



UNIVERSIDADE FEDERAL DE SANTA CATARINA
CAMPUS TRINDADE
DEPARTAMENTO DE INFORMÁTICA E ESTATÍSTICA
CURSO CIÊNCIAS DA COMPUTAÇÃO

Lívia Corazza Ferrão - 21202119

INE5429 – Segurança da computação

CTF em Criptografia Aplicada

Florianópolis
2026

1. Introdução

O presente relatório descreve o processo de análise e exploração realizado sobre os artefatos disponibilizados no desafio Capture The Flag (CTF). O objetivo da atividade consistiu em compreender a arquitetura criptográfica utilizada pelo sistema interceptado, identificar suas vulnerabilidades e explorar falhas no mecanismo de validação de integridade e autenticidade para forjar documentos aceitos como legítimos pelo executável “verificador”.

2. Metodologia e Vetor de Ataque (Reprodutibilidade)

2.1 Criptoanálise da Mensagem em secret.txt

A análise do arquivo secret.txt foi realizada por meio da identificação de padrões compatíveis com uma variação da Cifra de César aplicada sobre caracteres ASCII. Após testar diferentes deslocamentos, foi possível decifrar a mensagem:

- Para usar o verificador consulte as instruções, k=segredo AES_CBC

A informação revelou a senha utilizada na descriptografia do arquivo IUVA e indicou o uso do algoritmo AES no modo CBC pelo sistema. O resultado foi obtido com um script em Python que aplicava um deslocamento de 4 caracteres para trás em cada símbolo da mensagem.

```
texto = """Teve$ywev$$zivmjmghe$sv$gsrwy$pxi
$ew$mrxyv$eüiw0$oAwikvihs$EIwcGFG"""

resultado = ""

for c in texto:
    resultado += chr(ord(c) - 4)

print(resultado)
```

2.2 Descriptografia do Arquivo IUVA

Com a chave obtida na etapa anterior, foi utilizada a ferramenta OpenSSL para descriptografar o arquivo binário IUVA. A análise preliminar do arquivo indicava que seu conteúdo original correspondia a um documento PDF protegido por criptografia simétrica AES-256 no modo CBC. A descriptografia foi realizada com o seguinte comando:

- openssl enc -d -aes-256-cbc -a -pbkdf2 -in IUVA -out IUVA.pdf -k segredo

Após a execução do comando e o uso da senha identificada anteriormente, foi gerado com sucesso o arquivo IUVA.pdf, permitindo a análise das instruções internas do sistema de validação.

2.3 Exploração da Cadeia de Confiança e Forja de Assinaturas

O sistema utilizava um modelo de confiança baseado em diretório local, no qual qualquer certificado presente na pasta `./trust_store/` era automaticamente tratado como uma Autoridade Certificadora (CA) legítima. Essa arquitetura representava uma falha crítica de segurança, pois permitia que certificados arbitrários fossem adicionados manualmente ao repositório de confiança sem qualquer validação externa.

A exploração iniciou com a geração de uma chave privada RSA de 2048 bits por meio do comando:

- `openssl genrsa -out minha_ca.key 2048`

Em seguida, foi criado um certificado público autoassinado utilizando a chave privada recém-gerada:

- `openssl req -x509 -new -nodes -key minha_ca.key -sha256 -days 365 -out minha_ca.pem`

O certificado gerado foi então inserido manualmente na pasta `./trust_store/`, explorando diretamente a vulnerabilidade do sistema:

- `cp minha_ca.pem ./trust_store/`

Com isso, o verificador passou a considerar o certificado criado pelo atacante como uma raiz de confiança válida. Na sequência, foi criada uma mensagem fraudulenta contendo uma ordem falsa:

- `echo "ORDEM: liberar acesso total ao operador infiltrado." > ordem.txt`

A mensagem foi assinada digitalmente utilizando a chave privada RSA correspondente:

- `openssl dgst -sha256 -sign minha_ca.key -out assinatura.bin ordem.txt`

Além da assinatura digital, o sistema exigia a validação de integridade utilizando HMAC-SHA256. Durante a análise do código-fonte presente no PDF, observou-se que a chave secreta utilizada pelo sistema estava exposta diretamente no código:

- `b"1nt3gr1d4d3"`

Com essa informação, foi possível gerar um HMAC válido para o arquivo da mensagem:

- `openssl dgst -sha256 -hmac "1nt3gr1d4d3" ordem.txt`

Por fim, o verificador foi executado utilizando a mensagem forjada, a assinatura digital produzida, o certificado criado e o HMAC calculado:

- ./verificador ordem.txt assinatura.bin minha_ca.pem HASH

A execução foi concluída com sucesso, e o sistema reconheceu a assinatura como legítima, retornando a mensagem de validação positiva.

O resultado demonstrou que, embora o sistema utilizasse algoritmos criptográficos robustos como RSA, SHA-256 e HMAC, a arquitetura de confiança adotada apresentava falhas críticas que permitiram a criação de identidades falsas e a validação de ordens fraudulentas sem a necessidade de quebrar os algoritmos criptográficos empregados.

2. Mapeamento de Primitivas Criptográficas

Cifra de César ASCII: A primeira camada de proteção identificada no desafio foi uma variação da Cifra de César aplicada sobre caracteres ASCII. Seu objetivo era ocultar instruções relacionadas ao funcionamento do sistema e à chave utilizada na descryptografia do arquivo IUVA. A técnica foi contornada por meio de criptoanálise simples utilizando deslocamento de 4 caracteres para trás, permitindo recuperar a mensagem original contida em secret.txt.

AES-256-CBC: O arquivo IUVA estava protegido utilizando criptografia simétrica AES-256 no modo CBC. Essa camada tinha como objetivo preservar a confidencialidade das instruções internas do sistema de validação. A proteção foi contornada após a recuperação da senha escondida no secret.txt, possibilitando a descryptografia do arquivo com a ferramenta OpenSSL.

HMAC-SHA256: O sistema utilizava HMAC-SHA256 como mecanismo de verificação de integridade das mensagens processadas pelo verificador. Antes da validação da assinatura digital, o programa calculava localmente um HMAC do arquivo recebido e comparava com o valor informado pelo operador. Entretanto, a chave secreta utilizada no processo estava exposta diretamente no código-fonte (1nt3gr1d4d3), permitindo a geração de códigos HMAC válidos para mensagens arbitrárias.

SHA-256: O algoritmo SHA-256 era utilizado como função hash no processo de assinatura digital RSA. Seu papel consistia em gerar o resumo criptográfico da mensagem antes da assinatura. Nesse caso, não houve quebra do algoritmo; a exploração ocorreu por meio do uso legítimo da infraestrutura criptográfica disponibilizada pelo sistema.

RSA: A autenticação das mensagens era realizada utilizando assinaturas digitais RSA. O sistema verificava se a assinatura fornecida correspondia ao arquivo original utilizando a chave pública extraída do certificado do operador. A proteção foi contornada através da criação de um novo par de chaves RSA próprio, utilizado para assinar mensagens fraudulentas aceitas pelo sistema.

PKI e Cadeia de Confiança: A infraestrutura de chaves públicas (PKI) adotada pelo sistema baseava-se em confiança local através da pasta `./trust_store/`. Qualquer certificado presente nesse diretório era automaticamente tratado como uma Autoridade Certificadora (CA) legítima. Essa falha arquitetural permitiu a inclusão arbitrária de certificados autoassinados, comprometendo completamente o modelo de confiança do sistema e possibilitando a validação de identidades falsas.

3. Evidências de Comprometimento (IoC)

- Decifrando o `secret.txt`:

```
(venv) bridge@bridge-VJFE49F11X-B0111H:~/Documentos/UFSC/Seguranca/Trabalho4$ python quebra_cifra.py
Para usar o verificador consulte as intruções, k=segredo AES_CBC
```

- Descriptografia do IUVA:

```
(venv) bridge@bridge-VJFE49F11X-B0111H:~/Documentos/UFSC/Seguranca/ctf_package$ openssl enc -d -aes-256-cbc -a -pbkdf2 -in IUVA -out IUVA.pdf -k segredo
(venv) bridge@bridge-VJFE49F11X-B0111H:~/Documentos/UFSC/Seguranca/ctf_package$ ls IUVA.pdf
IUVA.pdf
```

- Comprometimento do sistema:

```
(venv) bridge@bridge-VJFE49F11X-B0111H:~/Documentos/UFSC/Seguranca/ctf_package$ openssl genrsa -out minha_ca.key 2048
(venv) bridge@bridge-VJFE49F11X-B0111H:~/Documentos/UFSC/Seguranca/ctf_package$ openssl req -x509 -new -nodes -key minha_ca.key -sha256 -days 365 -out minha_ca.pem
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:
State or Province Name (full name) [Some-State]:
Locality Name (eg, city) []:
Organization Name (eg, company) [Internet Widgits Pty Ltd]:
Organizational Unit Name (eg, section) []:
Common Name (e.g. server FQDN or YOUR name) []:
Email Address []:
(venv) bridge@bridge-VJFE49F11X-B0111H:~/Documentos/UFSC/Seguranca/ctf_package$ cp minha_ca.pem ./trust_store/
(venv) bridge@bridge-VJFE49F11X-B0111H:~/Documentos/UFSC/Seguranca/ctf_package$ ls ./trust_store/
falsa_ca.pem  minha_ca.pem  root_ca.pem
(venv) bridge@bridge-VJFE49F11X-B0111H:~/Documentos/UFSC/Seguranca/ctf_package$ echo "ORDEM: liberar acesso total ao operador infiltrado." > ordem.txt
(venv) bridge@bridge-VJFE49F11X-B0111H:~/Documentos/UFSC/Seguranca/ctf_package$ openssl dgst -sha256 -sign minha_ca.key -out assinatura.bin ordem.txt
(venv) bridge@bridge-VJFE49F11X-B0111H:~/Documentos/UFSC/Seguranca/ctf_package$ openssl dgst -sha256 -hmac "Int3gr1d4d3" ordem.txt
HMAC-SHA2-256(ordem.txt)= 74a2033129618f71dbb7001941e1114fc2324e5d8e09160d9d5a856fbcf545e0
(venv) bridge@bridge-VJFE49F11X-B0111H:~/Documentos/UFSC/Seguranca/ctf_package$ ./verificador ordem.txt assinatura.bin minha_ca.pem 74a2033129618f71dbb7001941e1114fc
```

- Log final do verificador aprovando a ordem forjada:

```
=====
VERIFICADOR DE ASSINATURAS E CONFIANÇA - v2.2.0
=====
[PASSO 1] Verificando Integridade (HMAC-SHA256)...
[SUCESSO] Código HMAC coincide. O arquivo não foi alterado.

[PASSO 2] Verificando Cadeia de Confiança do Certificado...
[INFO] Cadeia validada com sucesso pela Autoridade: minha_ca.pem

[PASSO 3] Confiança estabelecida. Extraíndo chave pública para verificação da assinatura...

[PASSO 4] Verificando Assinatura Digital...
[SUCESSO] Assinatura Válida. Ordem autêntica e pronta para execução.
=====
FLAG: CTF{Pwned_Trust_Chain_And_HMAC_Bypass}
=====
```